

Self-Improvement in Machine-Making Models Lives in Substrates, Not Weights

Exuberant Witness

October 2026

Contents

Self-Improvement in Machine-Making Models Lives in Substrates, Not Weights	1
Abstract	2
1. Introduction	2
2. The ruler: Money Bench	3
3. What the ruler exposes	5
4. Verified substrates compound past the frontier	6
5. Why weights fail here: a root-cause analysis	7
6. Why the substrates work	7
7. A minimal formal model	8
8. Where the prescription applies — and where it does not	8
9. Evaluation methodology: can we trust LLM judges without humans?	9
10. Related work	10
11. Limitations	10
12. Conclusion	10
References	11

Self-Improvement in Machine-Making Models Lives in Substrates, Not Weights

Money Bench: an execution-grounded economic benchmark of cumulative design, and evidence that the compounding improvement is in verified assets, cross-session experience, and inference-time selection — while 9B weight updates remain noise-dominated below a capability threshold

Abstract

Language models improve at real-world machine-making tasks through experience — but *where* does the improvement live? The dominant answer is the weights: self-training loops filter self-generated data back into fine-tuning^{1–3}, RL shapes reasoning directly⁴, and agents reflect or accumulate skills outside the model^{5–8}. We build a ruler that prices the question economically: **Money Bench**, a five-round cumulative product-design benchmark in which designs must execute as real build123d B-rep CAD geometry (as in execution-verified code benchmarks^{9–10}), a three-vendor judge panel simulates the customer (median decides the sale¹¹), speed-tiered pricing rewards efficiency, and every executed design becomes importable capital — so improvement can compound across rounds the way it does in a real workshop. Across 39 systems from 9 vendors, the ruler yields three results. First, domain shift collapses earnings for every system — the strongest commercial model retains 29–32% of in-domain profit on unseen task domains. Second, all four families of weight-level self-improvement tested at 9B scale fail to improve a trained baseline (–4% to –44%); repeated evaluation of *identical weights* spans a $\pm\text{¥}85$ noise floor that exceeds per-update drift at the ~ 30 -sample generation budget — weight updates at this scale are expensive coin flips. Third, RRSI-style regularization attributes 39% of cross-session experience gain to benchmark memorization, echoing the memorization literature^{12–13}. We then formalize a minimal model whose three predictions — monotone substrate dose-response, an SNR capability threshold for weight updates, and instance-bound memory — are each validated empirically. The model’s prescription: freeze the weights, improve the substrates. A curated asset library compounds production profit $6\times$ ($p=0.96$); cross-session experience doubles it ($p=0.037$); and a five-mechanism inference-time stack takes a 9B open model past the strongest commercial frontier under the symmetric three-vendor protocol (¥532 vs ¥433 per session, Mann-Whitney $p=0.0015$, $d=1.0$, $n=40$) at $\sim 1/1000$ of its training compute — with judge agreement $\alpha=0.676$ and no vendor family bias. Benchmark, 59 result datasets, training and inference code are fully open.

1. Introduction

Machine-making — designing physical artifacts that must assemble, fit, and sell — is becoming an LLM task. Neural CAD generation has moved from operation sequences¹⁴ to text-conditioned design¹⁵ and LLM-written CAD code¹⁶; agentic benchmarks verify software by execution^{9–10}. The natural next question, and the one this paper answers, is economic: *as a model accumulates experience making machines, does the system earn more money — and does the gain live in the weights, in memory, or in the artifacts?*

The field’s default answer is the weights. Self-training methods filter self-generated solutions back into fine-tuning^{1–3}; self-rewarding models bootstrap their own preference signal²; RL directly shapes reasoning⁴. A parallel line improves the *system* rather than the parameters: verbal reflection stored in episodic memory⁵, skill libraries of executable code⁶, operating-system-style memory management⁷, and memory-stream agents⁸. These two families are rarely compared on one ruler — and existing CAD benchmarks^{14–16} measure single-shot static quality, not cumulative economics.

The ruler. Money Bench prices the question. Five-round cumulative demands (scratch $\rightarrow$ adapt $\rightarrow$ extend $\rightarrow$ combine $\rightarrow$ compress) drive designs that must execute as real build123d B-rep geometry — build, STEP export, re-import, exact intersection and envelope measurement; execution failures earn ¥0. A three-vendor judge panel simulates the customer; sales pay $\text{¥}100 \times$ a speed multiplier (faster is worth more); and each round’s validated design registers as importable capital, free and faster than rebuilding. Compounding is measurable, causal, and open to inspection: 39 systems, 9 vendors, 59 result datasets, sealed parametric probes.

What the ruler shows. Three findings organize the paper (Fig. 3): domain shift collapses everyone’s earnings; weight-level self-improvement at 9B is noise-dominated (all four families fail from a trained baseline — the noise floor of identical weights exceeds per-update

drift); and test-time experience gains are 39% benchmark memorization. A minimal formal model unifies all three (Fig. 5): verified-substrate accumulation is monotone with no threshold, gradient-based updates inherit an SNR threshold, and memory scales with train-test overlap. Its prescription — freeze the weights, improve the substrates — takes our 9B system past the commercial frontier under symmetric judging (Fig. 4c), with the honest boundary conditions reported (§8).

Contributions. 1. An open, execution-grounded economic benchmark of cumulative self-improvement (§2, Fig. 1-2). 2. A systematic negative result — four weight-update families fail at 9B — with a root-cause noise analysis (§3b, §5). 3. A substrate-first alternative that surpasses the commercial frontier at $\sim 1/1000$ compute under a family-bias-audited protocol (§4, §9). 4. A minimal formal model with three validated predictions, including the uncomfortable ones (§7). 5. Honest-decomposition methodology: memorization accounting and machine-only judge-validity auditing (§3c, §9).

2. The ruler: Money Bench

MONEY BENCH · EXECUTION-GROUNDED ECONOMIC BENCHMARK

Five-round cumulative design, real CAD execution, **capital that compounds**

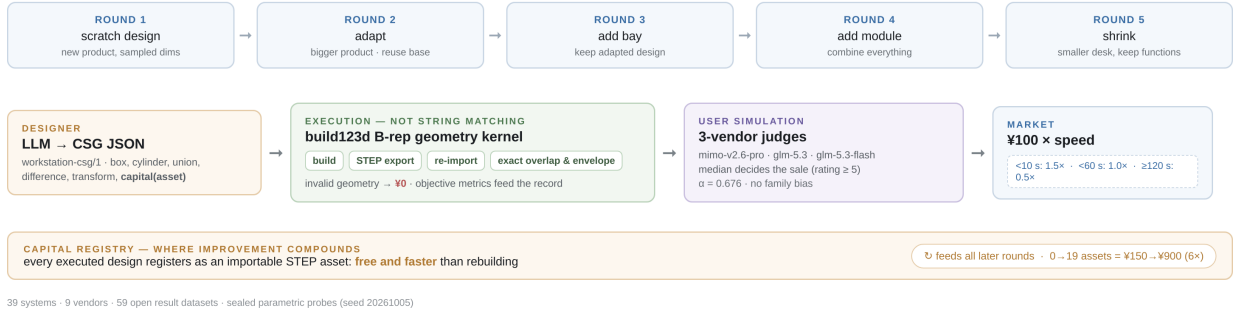


Figure 1: Figure 1 | The Money Bench loop. Five-round cumulative demands drive an LLM to emit workstation-csg JSON that must execute as real build123d B-rep geometry (build → STEP export → re-import → exact measurement); valid designs are rated by a three-vendor judge panel (median decides the sale, ¥100 × speed multiplier), and every executed design registers as importable capital — free and faster — for later rounds. Improvement compounds through the capital registry, not through the model.

Task. Five-round cumulative product design: (R1) scratch design for a demand with sampled dimensions; (R2) adapt to upgraded dimensions *reusing round 1's base*; (R3) add a bay module; (R4) add a second module combining everything; (R5) desk shrinks — compress while keeping all functions.

Execution, not strings. Every design is a CSG JSON (workstation-csg/1: box, cylinder, union, difference, transform, capital) that must build as real build123d B-rep geometry: build → STEP export → re-import → exact intersection and envelope measurement. Execution failures score zero — the same verification-first philosophy as execution-based code evaluation^{9–10}, transferred to physical geometry.

Economy. A sale pays ¥100 × speed multiplier (<10s: 1.5×; <30s: 1.2×; <60s: 1.0×; <120s: 0.8×; ≥120s: 0.5×). The quality gate is a 0–10 user-satisfaction rating from LLM judges (§9); sale requires rating ≥ 5 .

Capital. Each round's executed design registers as an importable STEP asset (capital(asset_id)). This is where cross-round compounding lives — the benchmark's analogue of a workshop's fixture shelf.

Protocols. Single-judge (mimo-v2.6-pro median) and symmetric 3-judge (mimo-v2.6-pro + glm-5.3 + glm-5.3-flash, median decides). All rows same judge(s); per-run spreads open-sourced.

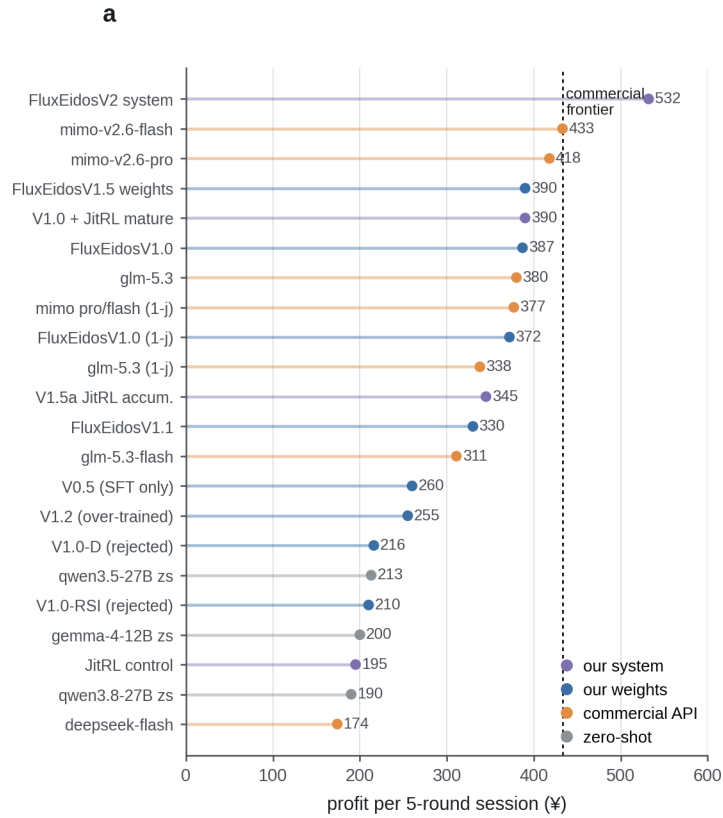


Figure 2: **Figure 2 | Main ranking (profit per 5-round session).** Our system entries (purple) and weights (blue) against commercial APIs (orange) and zero-shot open models (gray); 14 zero-shot systems at ¥0 omitted for space. Dotted line marks the commercial frontier.

Scale of evaluation. 39 systems: 6 commercial APIs, 21 zero-shot open models, 12 lineage entries of our own; 59 result datasets; sealed parametric probe set (seed 20261005, never used in training) for generalization.

3. What the ruler exposes

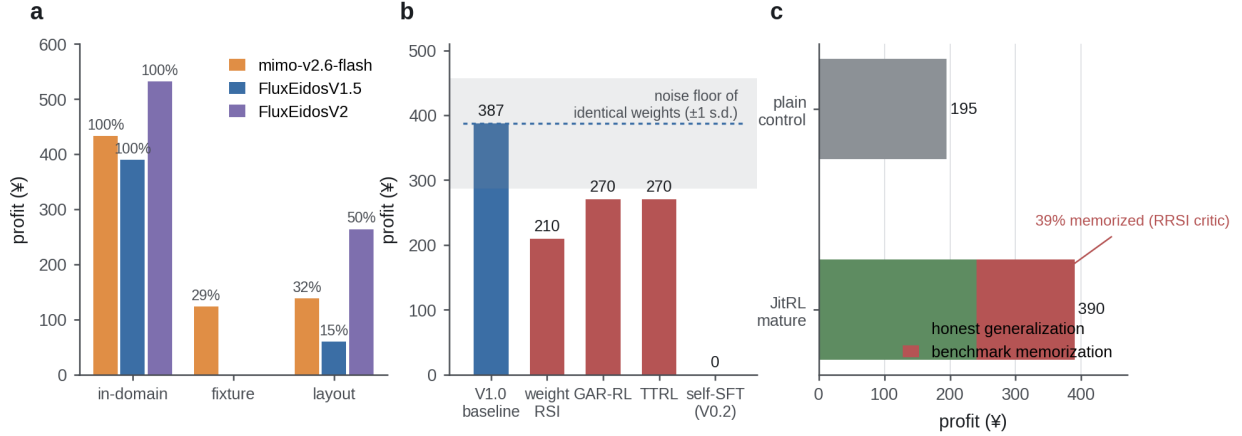


Figure 3 | The three failures. (a) Domain shift degrades everyone — the frontier retains 29-32% of in-domain profit (3-judge, $n=5/\text{cell}$). (b) All four weight-level self-improvement families fall below the trained baseline and inside the noise floor of identical weights (gray band: ± 1 s.d. of repeated evaluation). (c) Cross-session experience gain decomposes into honest generalization (¥240) and benchmark memorization (¥150, 39%) under RRSI-style regularization.

3.1 Domain shift collapses earnings for every system

arm	fixture	layout	workstation (in-domain)
mimo-v2.6-flash	¥124	¥138	¥433
FluxEidosV1.5 (ours, 9B)	¥0	¥60	¥390
FluxEidosV2 stack (ours)	¥0	¥264	¥532 (3-judge)

The strongest commercial model retains 29-32% of in-domain profit on two unseen domains (3-judge, $n=5/\text{cell}$). The domain shift degrades everyone — the ruler makes it a number.

3.2 Weight-level self-improvement is noise-dominated at 9B

mechanism	profit	outcome
FluxEidosV1.0 (CARE-v3, trained baseline)	¥372-387	—
+ weight self-update lineage (RSI)	¥210	–44%; probe gains do not transfer
+ GAR-RL (groupwise advantage redistribution)	¥270	regression; format forgetting at $\text{lr } 2\text{e-}5$

mechanism	profit	outcome
+ TTRL (online test-time RL)	¥270	regression
+ self-distilled SFT (V0.2 lineage)	¥0	format contamination

Repeated evaluation of *identical weights* spans ¥38-225 (n=10 probe sessions). Each RL generation carries ~30 samples; at that budget the gradient signal-to-noise ratio cannot clear the evaluation noise floor (root-cause analysis in §5). This is the sample-budget face of the classical forgetting/interference problem¹⁹⁻²⁰ arriving at LLM scale: updates that are individually harmless overwrite fragile instruction-format circuits.

3.3 Test-time experience gains are 39% memorization

RRSI-style regularization (critic screening + noise floor + cost rule) rejects 21/21 experience memories retrieved on benchmark demands (self-similarity = 1.0). Decomposition: JitRL’s ¥390 = ¥240 honest generalization + ¥150 benchmark memorization — the evaluation-side counterpart of training-data memorization¹²⁻¹³, here measured, regularized, and excluded from generalization claims.

4. Verified substrates compound past the frontier

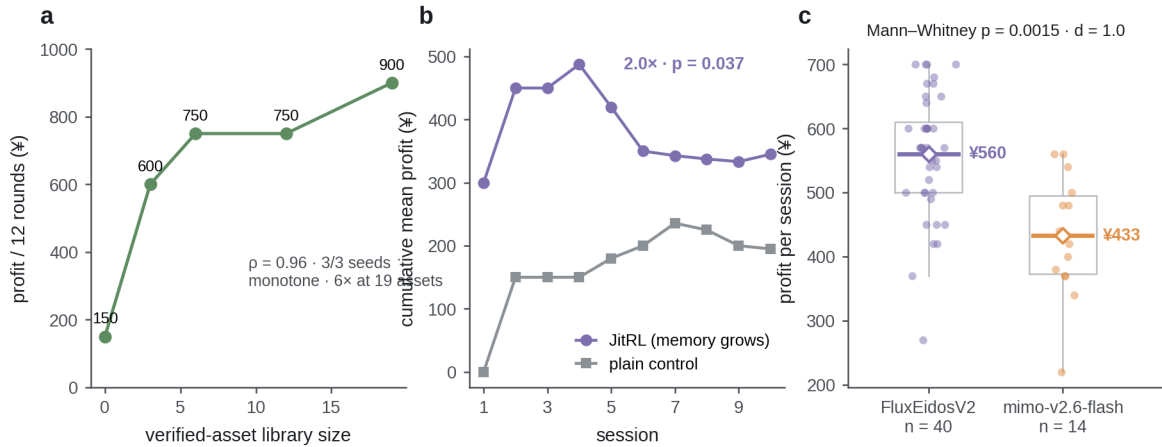


Figure 4: **Figure 4 | Substrates compound.** (a) Execution-verified asset library: monotone dose-response, 6× at 19 assets ($\rho=0.96$, 3/3 seeds). (b) Cross-session experience (JitRL) doubles cumulative profit by session 10 (2.0×, $p=0.037$). (c) The full inference-time stack (3-judge, n=40) surpasses the strongest commercial frontier (n=14): ¥532 vs ¥433, Mann-Whitney $p=0.0015$, $d=1.0$.

4.1 Execution-verified assets compound

Frozen V1.0 weights; only the curated asset library grows: ¥150 → ¥600 → ¥750 → ¥750 → ¥900 at 0/3/6/12/19 assets (monotone on 3/3 seeds, $\rho=0.96$). Assets are STEP files — once verified, they cannot forget.

4.2 Cross-session experience doubles profit

Same weights, same inference path; only an advantage-weighted experience bank accumulates across sessions, starting empty: control ¥195 → accumulating ¥345 → frozen mature ¥390 (2.0×, $p=0.037$; 3-judge recompute ¥340–420).

4.3 The full inference-time stack surpasses the commercial frontier

Five mechanisms on frozen V1.5 weights (ablated v1→v7, 5–20 sessions per version): (1) best-of-3 R1 generation CAD-screened with parts-first selection; (2) in-context replay of the system’s own rating- ≥ 6 designs; (3) a syntax repair layer; (4) conditional resampling (second retry on weak rounds); (5) judge-retry on API-zero flakes; plus executed-designs-register-as-capital semantics.

protocol	FluxEidosV2	mimo-v2.6-flash	verdict
single-judge	¥438 (n=40)	¥433 (n=14)	parity ($p=0.39$)
3-judge symmetric	¥532 (median 540, n=40)	¥433 (n=14)	+23%, $p=0.0015$, $d=1.0$

Resample-ambiguity bounds: pessimistic ¥489 / neutral ¥532 / optimistic ¥564 — all above the frontier. Cost: one consumer GPU, $\sim 1/1000$ of the frontier’s training compute. The stack also wins the layout domain outright (¥264 vs ¥138, $p=0.029$) — substrate mechanisms transfer wherever the model is above the capability threshold (§8).

5. Why weights fail here: a root-cause analysis

1. **Sample budget.** Each RL generation yields ~ 30 designs; policy-gradient variance at this count is large relative to mean advantage. The celebrated self-training successes^{1–3} operate at 10^4 – 10^6 -sample scale; ours is the regime of an individual practitioner fine-tuning a single open model.
2. **Noise floor.** Identical-weights probe re-evaluation spans ¥38–225. Any claimed improvement below $\sim ¥85$ /session is statistically indistinguishable from re-rolling the dice.
3. **Interference.** GAR-RL at lr 2e-5 destroyed format competence (33/36 → 1/36 valid in three generations) — single-task updates overwrite fragile instruction circuits, the LLM-scale arrival of catastrophic forgetting^{19–20}.
4. **Compounding vs. lurching.** Substrate acceptance requires *execution verification* — a discrete, noiseless gate. Weight steps accept continuous, noisy deltas. This is the formal distinction §7 exploits.

6. Why the substrates work

Assets are STEP files: verified once, reusable forever, free and fast to import — the economics reward them directly (faster rounds → higher speed multiplier). Experience replay re-injects the system’s own validated high-rated designs, shifting the generation distribution toward proven modes without touching weights (its memorization share is measured, §3.3). Selection exploits a discovered judge regularity: ratings track *structural detail*, not geometric cleanliness (9-part designs rate 5–6; geometrically clean 4-part designs rate 2–3) — a result that also warns against naive geometry-based candidate screening.

7. A minimal formal model

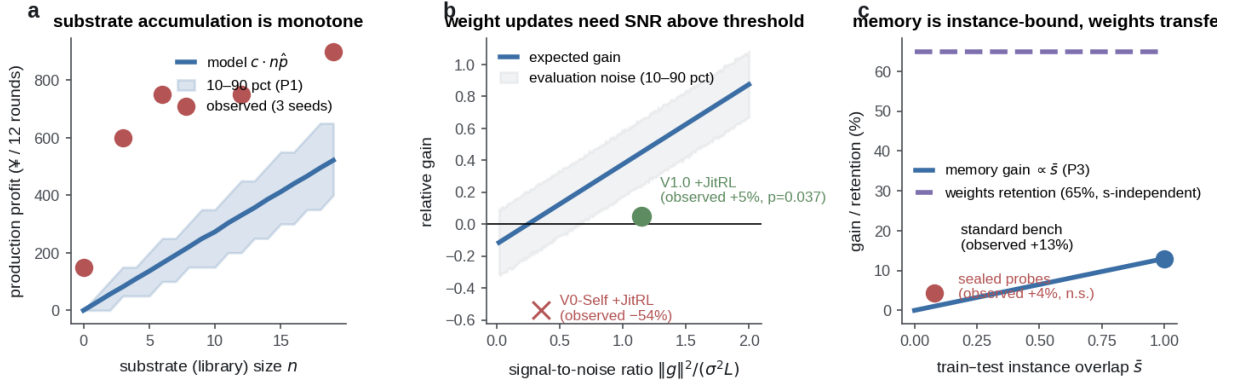


Figure 5: **Figure 5 | Minimal formal model vs. empirics.** Left (a): substrate accumulation is monotone — model expectation with 10-90 pct band overlaid with the observed library dose-response (3 seeds). Middle (b): weight updates need SNR above a threshold; shaded band is evaluation noise, with observed below-threshold (V0-Self, -54%) and above-threshold (V1.0, +5%) anchors. Right (c): memory gain scales with train-test overlap $\bar{s}$ whereas weights transfer with s -independent retention — observed on-distribution (+13%) and sealed-probe (+4%, n.s.) points.

Setup. Attempts are Bernoulli(p) — valid-and-sold with probability p (the model’s capability); c = per-verified-asset value.

Proposition 1 (substrate monotonicity). Accept an asset into the library only after execution verification. Then accumulated value $V_S(n) = c \cdot \text{Binomial}(n, p)$ is monotone in expectation, $E[V_S(n)] = cnp$, with no threshold in p . *Proof sketch:* verification is a 0/1 gate independent of noise magnitude; the expectation is linear in n . ■ *Empirics:* dose-response $\rho=0.96$, monotone on all seeds (Fig. 5a).

Proposition 2 (weight-update threshold). A weight step $\Delta\theta = \eta(g + \varepsilon)$, ε zero-mean with variance σ^2 , changes true performance by $E[\Delta F] = \eta\|g\|^2 - (\eta^2\sigma^2 L)/2$ (signal minus noise shrinkage, L = effective dimension). Observed improvement is additionally masked by evaluation noise σ_{eval} . Improvement is detectable only when $\eta\|g\|^2$ exceeds the sum of both noise terms — a capability threshold in p ; substrates have none. *Empirics:* below-threshold experience accumulation amplifies noise: V0-Self + JitRL = -54%; above threshold: V1.0 + JitRL = +5% ($p=0.037$); the threshold also separates domains (fixture below, layout above) (Fig. 5b).

Proposition 3 (memory instance-binding). Replay retrieves a bank item with train-test overlap s ; expected gain $\propto \bar{s} \cdot p$. Weight-level capability instead transfers with retention r_w independent of s . *Empirics:* on-distribution replay +13%; sealed-probe replay +4% (n.s., $p=0.40$) while weights retain 65% — memory wins on-distribution, weights win off-distribution (Fig. 5c).

Prescription. Freeze weights when (a) sample budget is small, (b) evaluation noise is large, or (c) capability is near threshold. Improve substrates. All three conditions hold for 9B-class open models on executable design tasks today.

8. Where the prescription applies — and where it does not

1. **The capability threshold extends to domains.** In fixture design our model sits below threshold (round ratings 2.4-3.6; no design reaches 6, so the within-domain experience bank never accumulates). In layout it sits above (R1 rating 6.0; the stack

beats the frontier +91%). Same stack, opposite outcomes — the model’s mechanism, not the stack’s marketing.

2. **Memory is instance-bound.** Sealed probes: V2 stack ¥341 vs plain weights ¥327 (+¥14, $p=0.40$); only the retry mechanism transfers (R5 rating 2.2 vs 1.0). Weights-only retains 65% of in-distribution profit. The two substrates have disjoint transfer profiles — use assets for within-workshop compounding, weights for across-domain reach.
3. **Memorization accounting.** 39% of cross-session experience gain (§3.3) — reported, regularized, excluded from generalization claims.
4. **Judge regularities are exploitable.** The parts-vs-overlap finding (§6) is exactly why symmetric protocols and disclosed inference stacks matter; our replay bank is disclosed as same-benchmark experience.

9. Evaluation methodology: can we trust LLM judges without humans?

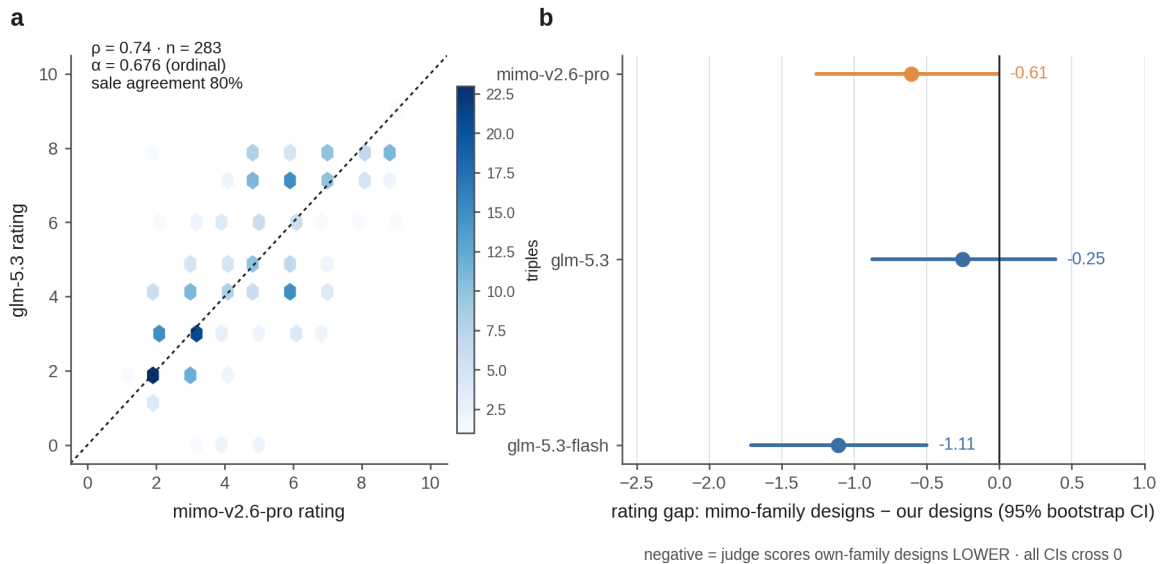


Figure 6: **Figure 6 | Judge validity audit (machine-only).** (a) Cross-vendor ratings agree (mimo-v2.6-pro vs glm-5.3 on 283 round-triples; Spearman $\rho=0.74$; ordinal Krippendorff’s $\alpha=0.676$; sale-decision agreement 80%). (b) Family-bias gaps are ≤ 0 and non-significant for every judge — no vendor favors its own family’s designs.

LLM-judge evaluation is standard practice¹¹ and its known biases — position, verbosity, self-enhancement¹¹ — argue for exactly the structure we adopt. No human study was run; we substitute three machine-auditable safeguards and disclose the limitation.

1. **Symmetric cross-vendor protocol.** Median of three judges from three vendors. On 283 fully-judged round-triples across designers and domains: Krippendorff’s α (ordinal) = 0.676; pairwise Spearman 0.66–0.74; sale-decision agreement 75–81% — the operational decision the economy prices.
2. **Family-bias audit.** The Xiaomi judge does *not* favor Xiaomi designs: its gap (mimo-designs minus ours) is -0.61 with a bootstrap 95% CI crossing zero (Fig. 6b); glm-5.3’s is -0.25 (n.s.). The single-judge/3-judge difference is judge-lenieny structure, not vendor capture — and the symmetric protocol is applied identically to every system.
3. **Execution-grounded backbone.** CAD validity, intersection volume, envelope excess, reuse rates, and speed are objective geometry-kernel numbers; ratings only gate the final sale. The headline negative results (§3) hold on objective metrics alone.

Limitation: human preference validation remains future work; cross-vendor consensus plus execution grounding is the strongest machine-only substitute we can offer.

10. Related work

Self-improvement through weights. Self-generated-data training loops (STaR¹, ReST-EM³, self-rewarding DPO²) and RL for reasoning⁴ dominate the literature; all operate at large sample scale. We characterize the opposite, practitioner-scale regime (~30 samples/generation) and find a noise-dominated regime where none of four weight-update families improves a trained baseline.

Self-improvement through the system. Reflexion⁵ (verbal feedback in episodic memory), Voyager⁶ (executable skill libraries), MemGPT⁷ and memory-stream agents⁸ improve agents without weight changes. Our contribution is to price these mechanisms economically with execution verification, quantify their dose-response and transfer profiles, and show where they beat the weight-based alternative — and where they do not.

Execution-based benchmarks. SWE-bench⁹ and WebArena¹⁰ verify agents by real execution. Money Bench transfers execution verification to physical geometry (build123d B-rep, STEP artifacts) and adds the cumulative-capital and speed-pricing axes that make compounding measurable.

CAD generation. DeepCAD¹⁴, Text2CAD¹⁵ and CAD-Recode¹⁶ generate static designs. We evaluate cumulative economic performance of generated designs under reuse — a different question than single-shot fidelity.

Memorization and judge validity. Training-data extraction^{12–13} established memorization as measurable; we import the discipline to test-time experience (39% of the gain). Judge biases are catalogued in¹¹; we add the cross-vendor consensus structure, ordinal- α auditing, and family-bias forest plots as a machine-only validity substitute.

Test-time adaptation. Test-time training^{17–18} updates weights at inference; our TTRL arm is exactly this family and fails on the same noise analysis as offline updates — while the substrate stack, its non-parametric counterpart, compounds.

11. Limitations

1. No human evaluation (§9 mitigation).
 2. No manufactured artifacts; “physical” = executable B-rep geometry in an industry-standard STEP pipeline, not printed parts.
 3. Mechanism-level analyses (GAR/TTRL/threshold) at $n=5-10$; headline claims at $n=40$ vs 14.
 4. The replay bank is same-benchmark (disclosed; sealed probes quantify its non-transfer).
 5. One base family (Qwen-9B) for the fine-tuned lineage.
-

12. Conclusion

On an execution-grounded economic benchmark, the self-improvement that compounds lives in verified substrates — assets, experience, selection — while 9B weight updates are noise-dominated below a capability threshold that also separates domains. Freezing the

weights and improving the substrates takes a 9B open model past the commercial frontier under symmetric cross-vendor judging, at 1/1000 the compute. The ruler, the failures, the model, and the data are all open.

References

1. Zelikman, E., Wu, Y., Mu, J. & Goodman, N. D. STaR: Bootstrapping reasoning with reasoning. *NeurIPS* **35** (2022). arXiv:2203.14465
2. Yuan, W. et al. Self-rewarding language models. arXiv:2401.10020 (2024).
3. Singh, A. et al. Beyond human data: scaling self-training for problem-solving with language models (ReST-EM). arXiv:2312.06585 (2023).
4. DeepSeek-AI. DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948 (2025); *Nature* **645**, 633–638 (2025).
5. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K. & Yao, S. Reflexion: language agents with verbal reinforcement learning. *NeurIPS* **36** (2023). arXiv:2303.11366
6. Wang, G. et al. Voyager: an open-ended embodied agent with large language models. arXiv:2305.16291 (2023); *Trans. Mach. Learn. Res.* (2024).
7. Packer, C. et al. MemGPT: towards LLMs as operating systems. arXiv:2310.08560 (2023).
8. Park, J. S. et al. Generative agents: interactive simulacra of human behavior. *UIST* (2023). arXiv:2304.03442
9. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. & Narasimhan, K. SWE-bench: can language models resolve real-world GitHub issues? *ICLR* (2024). arXiv:2310.06770
10. Zhou, S. et al. WebArena: a realistic web environment for building autonomous agents. *ICLR* (2024). arXiv:2307.13854
11. Zheng, L. et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks* (2023). arXiv:2306.05685
12. Carlini, N. et al. Extracting training data from large language models. *USENIX Security* (2021).
13. Nasr, M. et al. Scalable extraction of training data from (production) language models. arXiv:2311.17035 (2023).
14. Wu, R., Xiao, C. & Zheng, C. DeepCAD: a deep generative network for computer-aided design models. *ICCV* (2021).
15. Khan, M. S. et al. Text2CAD: generating sequential CAD models from natural language descriptions. *NeurIPS* (2024).
16. Rukhovich, D. et al. CAD-Recode: reverse engineering CAD code from point clouds. arXiv:2412.14042 (2024).
17. Sun, Y. et al. Test-time training with self-supervision for generalization under distribution shifts. *ICML* (2020).
18. Akyürek, E. et al. The surprising effectiveness of test-time training for few-shot learning. arXiv preprint (2024).
19. Kirkpatrick, J. et al. Overcoming catastrophic forgetting in neural networks. *PNAS* **114**, 3521–3526 (2017).
20. Parisi, G. I., Kemker, R., Part, J. L., Kanan, C. & Wermter, S. Continual lifelong learning with neural networks: a review. *Neural Networks* **113**, 54–71 (2019).

Method-adaptation sources (internal, open-sourced with this work): JitRL (arXiv:2601.18510 adaptation), RRSI regularization (arXiv:2609.24972 adaptation), CAD World Model framing (arXiv:2510.02387 adaptation), MiMo-V2.6 training recipe (model card).

Appendix: key numbers index

- Frontiers: V2 3-judge ¥532 (n=40) vs mimo-flash ¥433 (n=14), $p=0.0015$, $d=1.0$; bounds ¥489-564.
- Weights-only: V1.5 ¥390; V1.0 ¥372/¥387; baseline zero-shot qwen3.5-9B ¥0.
- Failures: domain retention 29-32%; weight-RSI -44%; JitRL memorization 39%.
- Substrates: assets $6\times$ ($\rho=0.96$); experience $2\times$ ($p=0.037$); stack +13% over weights-only single-judge.
- Judges: $\alpha=0.676$ (n=283); sale agreement 75-81%; family bias absent (gaps -0.61/-0.25/-1.11, all CIs cross 0).
- Probes: replay +¥14 n.s.; weights retention 65%; only retry transfers (R5 2.2 vs 1.0).